**Chapter 6: Pipelining covers:**

1. What pipelining is and how it improves instruction throughput
2. Pipeline stages in MIPS: IF, ID, EX, MEM, WB
3. Pipeline hazards: structural, data, and control
4. Solutions to hazards: forwarding, stalling, branch prediction, reordering
5. Pipelined datapath design
6. Advanced topics: superscalar execution, dynamic scheduling, speculative execution

## 📘 Chapter 6: Pipelining – Enhancing Performance via Parallelism

### 🔹 Slide / Section: Introduction to Pipelining

### 📌 Summary:

- Pipelining increases throughput, not individual instruction speed.
- Like an assembly line: multiple instructions are processed simultaneously at different stages.

### 💡 Teaching Points:

- Use the laundry analogy from the slides:
  - Washing, drying, and folding clothes in parallel → faster overall completion.
- Emphasize that pipelining doesn't reduce the latency of one instruction, but improves the throughput of many.

### 🎯 Student Questions:

- "Why doesn't pipelining make each instruction run faster?"
- "Can you think of another real-life example of pipelining?"

### 🔹 Slide / Section: Pipelined vs Single-Cycle Execution

| Feature | Single-cycle | Pipelined |
|---|---|---|

| | | |
|---|---|---|
| Clock cycle time | Long (slowest stage) | Short (max stage time + overhead) |
| Throughput | One instruction per cycle | Up to one instruction per cycle |
| Latency | Low per instruction | Higher due to pipeline fill/drain |

📌 **Summary:**

- Single-cycle wastes time; pipelined uses resources more efficiently.
- The example shows speedup from 8 ns/cycle to 2 ns/cycle with 5-stage pipeline.

🧠 **Key Insight:**

- Speedup ≈ the number of stages (ideal case).
- Unbalanced stages or pipeline overhead reduces actual speedup.

🔹 **Slide / Section: MIPS Pipeline Stages**

| Stage | Purpose |
|---|---|
| IF | Instruction Fetch |
| ID | Instruction Decode & Register Read |
| EX | ALU Operation / Address Calculation |
| MEM | Memory Access |

| | |
|---|---|
| WB | Write Back to Register |

📌 **Summary:**

- Each stage takes ~1 clock cycle.
- Every instruction goes through all stages, even if some don't use them (e.g., `j` skips ALU).

🎯 **Student Question:**

- "Which instruction might not use every stage? Why?"

🔹 **Slide / Section: Why Pipelining is Easier in MIPS**

✅ **Features That Help:**

- All instructions same length (32 bits)
- Few instruction formats
- Only `lw`/`sw` access memory
- Operands aligned in memory

❌ **Challenges:**

- Hazards can stall the pipeline:
  - Structural
  - Data
  - Control

🔹 **Slide / Section: Structural Hazards**

📌 **Summary:**

- Occurs when two instructions need the same hardware resource at the same time.

- Example: using single memory for both instruction and data → conflict between `lw` and next instruction fetch.

💡 **Solution:**

- Use separate instruction and data memories.

🎯 **Student Question:**

- "How could we avoid this hazard without separate memories?"

🔹 **Slide / Section: Control Hazards**

📌 **Summary:**

- Happens when a branch decision affects which instruction comes next.
- The PU may fetch the wrong instructions after a conditional branch.

💡 **Solutions:**

1. Stall the pipeline until the branch is resolved
2. Predict branch outcome (e.g., assume not taken)
3. Delay slots (not covered in detail)

🧠 **Diagram Tip:**

Show a pipeline bubble introduced due to misprediction.

🎯 **Student Question:**

- "What's the cost of a wrong branch prediction?"

🔹 **Slide / Section: Data Hazards**

📌 **Summary:**

- When an instruction needs data produced by a previous instruction still in the pipeline.

💡 **Types:**

- RAW (Read After Write) – most common
- WAR/WAW – rare in MIPS due to register write-back order

✅ **Full Forms:**

| Abbreviation | Full Form |
| --- | --- |
| RAW | Read After Write |
| WAR | Write After Read |
| WAW | Write After Write |

💡 **Solutions:**

1. Forwarding (bypass ALU result directly)
2. Stalling (insert bubbles)
3. Code reordering (software solution)

🧠 **Diagram Tip:**

Draw data forwarding paths from EX/MEM to EX stage.

🎯 **Student Question:**

- “Why is forwarding better than waiting for WB?”

🔹 **Slide / Section: Load-Use Hazard**

📌 **Summary:**

- Special case of RAW where a load is followed by an instruction that uses its result.
- Needs a bubble/stall unless forwarding is supported from MEM stage.

🧠 **Teaching Tip:**

Walk through:

```
lw $t0, 0($t1)   # Load word from memory into register $t0
add $t2, $t0, $t3 # Add contents of $t0 and $t3; store result in $t2
```

This is a classic example used to demonstrate data hazards, especially RAW (Read After Write) hazards. We'll explain how this executes in a pipelined MIPS processor, stage by stage.

→ Explain why a stall is needed before add.

🔹 **Slide / Section: Reordering Code to Avoid Stall**

📌 **Summary:**

- The compiler can reorder instructions to avoid hazards.
- Example:

```
lw $t0, 0($t1)     # Load word at address $t1 + 0 into $t0
lw $t2, 4($t1)     # Load word at address $t1 + 4 into $t2
sw $t0, 4($t1)     # Store $t0 to address $t1 + 4
```

```
sw $t2, 0($t1)     # Store $t2 to address $t1 + 0
```

→ Reordered:

```
lw $t0, 0($t1)     # Load word at address $t1 + 0 into $t0
lw $t2, 4($t1)     # Load word at address $t1 + 4 into $t2
sw $t2, 0($t1)     # Store $t2 to address $t1 + 0
sw $t0, 4($t1)     # Store $t0 to address $t1 + 4
```

🎯 **Student Task:**

Ask students to identify RAW hazards in the original code.

🔹 **Slide / Section: Pipelined Datapath Design**

📌 **Summary:**

- The pipelined datapath uses intermediate registers (pipeline registers):
  - IF/ID, ID/EX, EX/MEM, MEM/WB
- These store intermediate results between stages.

The pipelined datapath uses intermediate registers , also known as pipeline registers , to store and pass data between stages of the pipeline. These registers are:

- IF/ID: Between Instruction Fetch and Instruction Decode
- ID/EX: Between Instruction Decode and Execute
- EX/MEM: Between Execute and Memory Access
- MEM/WB: Between Memory Access and Write Back

They ensure that each stage of the pipeline can work independently on different instructions at the same time.

### 🔄 The 5 Stages of the MIPS Pipeline

| Stage | Purpose |
|---|---|
| IF (Instruction Fetch) | Fetch instruction from memory |
| ID (Instruction Decode & Register Read) | Decode instruction, read register values |
| EX (Execute / Address Calculation) | Perform ALU operations or compute memory addresses |
| MEM (Memory Access) | Load or store data from/to memory |
| WB (Write Back) | Write the result back to the register file |

Each stage works in parallel with the others, but needs its input data.

### 📦 Key Pipeline Registers and What They Store

| Pipeline Reg | From Stage | To Stage | Stores |
|---|---|---|---|
| IF/ID | IF | ID | Instruction fetched and updated PC+4 |
| ID/EX | ID | EX | Register values, immediate values, and control signals |

| | | | |
|---|---|---|---|
| EX/MEM | EX | MEM | ALU result, register destination, memory address |
| MEM/WB | MEM | WB | Data from memory or ALU, register destination |

🧠 **Key Point:**

- Each pipeline stage has its copy of control logic and data path.

🎯 **Student Question:**

- “Why do we need these intermediate registers?”

🔹 **Slide / Section: Bug in the Datapath**

📌 **Summary:**

- The early version had a bug where the destination register number came from a later instruction due to improper routing.
- Fixed by adding correct forwarding paths and ensuring register numbers flow properly through pipeline stages.

🧠 **Teaching Tip:**

Highlight the importance of correct signal routing in hardware design.

🔹 **Slide / Section: Superscalar and Dynamic Pipelining**

📌 **Summary:**

- Superscalar: executes multiple instructions per cycle
- Dynamic scheduling: allows out-of-order execution with in-order commit
- Combined with branch prediction = speculative execution

💡 **Advanced Concepts:**

- Out-of-order execution
- Speculative execution
- Commit unit (for undoing mispredicted branches)

🎯 **Student Question:**

- “Why is speculative execution risky but useful?”

✅ **Summary Table: Pipeline Hazards and Solutions**

| Hazard Type | Cause | Solution |
|---|---|---|
| Structural | Resource conflict | Separate hardware units |
| Data (RAW) | Dependency on prior result | Forwarding, stalling |
| Control | Branch misprediction | Prediction, delay slots |